

Chapter 1 — Basic Array Traversal

0. What You Will Learn in This Chapter

Basic Array Traversal is the foundation of almost every coding interview problem. Even advanced patterns like sliding window, prefix sum, greedy, DP, graph traversal, and monotonic stack all depend on one basic skill:

Move through the input carefully, maintain the right state, and update the answer at the right time.

By the end of this chapter, you should be able to solve problems where you need to:

- Scan an array once
- Track a maximum or minimum
- Count elements
- Maintain a running sum
- Track the best answer so far
- Compare neighboring elements
- Modify an array in-place
- Use a read/write pointer
- Traverse from right to left
- Build a result array
- Handle negative numbers, zeroes, duplicates, and overflow
- Explain $O(n)$ time and $O(1)$ space clearly
- Recognize when simple traversal is enough
- Recognize when another pattern is needed

This chapter may look easy, but do not underestimate it. Many medium problems are just advanced forms of:

“Scan once while remembering the right information.”

1. What Is Basic Array Traversal?

Basic Array Traversal means going through an array element by element and processing each value.

The simplest form is:

```
for (int i = 0; i < nums.size(); i++) {  
    // process nums[i]  
}
```

Or, if you only need the values and not the index:

```
for (int x : nums) {  
    // process x  
}
```

But the real skill is not writing the loop.

The real skill is knowing:

What information should I maintain while scanning?

For example:

Find maximum	maxVal
Find minimum	minVal
Count even numbers	count
Calculate total sum	sum
Find best stock profit	minPrice, bestProfit
Find maximum subarray	currentSum, bestSum
Move zeroes	write pointer
Check monotonic array	increasing, decreasing
Replace with greatest right value	maxRight

So Basic Array Traversal means:

Visit each element once and update a small amount of useful state.

2. Why This Pattern Exists

Many beginner solutions use unnecessary nested loops.

Example:

Find the maximum number in an array.

Bad brute force idea:

```
for each element x:
    check every other element to see if x is largest
```

That is unnecessary.

You only need one variable:

```
int maxVal = nums[0];

for (int x : nums) {
    maxVal = max(maxVal, x);
}
```

This reduces the problem from repeated checking to a single pass.

Basic traversal exists because many problems do not require storing everything or comparing everything.

They only require:

- current element
 - previous useful information
 - best answer so far
-

3. Core Need / Why Use This Pattern Over Others?

Use Basic Array Traversal when the problem can be solved with a simple scan and a few variables.

You should prefer basic traversal over heavier patterns when possible.

3.1 Why Not Sorting?

If the problem is only asking for maximum, minimum, count, or sum, sorting is usually unnecessary.

Example:

Find maximum element.

Sorting:

```
sort(nums.begin(), nums.end());  
return nums.back();
```

Time: $O(n \log n)$

Traversal:

```
int ans = nums[0];  
  
for (int x : nums) {  
    ans = max(ans, x);  
}
```

Time: $O(n)$

Traversal is better.

3.2 Why Not Hash Map?

Hash maps are useful when you need fast lookup or frequency counts for many values.

But if you only need one count, one sum, or one maximum, a hash map is unnecessary.

Example:

Count positive numbers.

Use:

```
int count = 0;  
  
for (int x : nums) {  
    if (x > 0) {  
        count++;  
    }  
}
```

No hash map needed.

3.3 Why Not Sliding Window?

Sliding window is for subarrays or substrings where you maintain a range.
If the problem only asks you to inspect each element independently, basic traversal is enough.

Example:

Find max consecutive ones can be solved by basic traversal because you only need a current streak.

But Max Consecutive Ones III, where you can flip at most k zeroes, needs sliding window.

3.4 Why Not Dynamic Programming?

DP is for choices and overlapping subproblems.

If there is no real choice structure and you only scan/update state, basic traversal is cleaner.

Example:

Maximum Subarray can be viewed as DP, but the optimized form is also a traversal pattern:

```
currentSum = max(nums[i], currentSum + nums[i]);  
bestSum = max(bestSum, currentSum);
```

4. When to Use Basic Array Traversal

Use this pattern when the problem asks you to do one of these.

4.1 Find Maximum or Minimum

Problem clues:

- “Find the largest...”
- “Find the smallest...”
- “Maximum value...”
- “Minimum value...”

Examples:

- Maximum element in array
 - Third Maximum Number
 - Largest Number At Least Twice of Others
 - Find the Highest Altitude
-

4.2 Count Elements

Problem clues:

- “Count how many...”
- “Return the number of...”
- “How many elements satisfy...”

Examples:

- Count even numbers
- Find Numbers with Even Number of Digits
- Count Items Matching a Rule
- Max Consecutive Ones

4.3 Running Sum / Total Sum

Problem clues:

- “Running sum...”
- “Cumulative...”
- “Total...”
- “Altitude after each gain...”

Examples:

- Running Sum of 1D Array
 - Find the Highest Altitude
 - Richest Customer Wealth
 - Find Pivot Index, though this becomes prefix-sum related
-

4.4 Track Best Answer So Far

Problem clues:

- “Maximum profit”
- “Maximum subarray”
- “Best score”
- “Largest possible...”

Examples:

- Best Time to Buy and Sell Stock
 - Maximum Subarray
 - Best Sightseeing Pair
 - Maximum Product Subarray
-

4.5 Compare Neighbors

Problem clues:

- “Sorted?”
- “Monotonic?”
- “Mountain?”
- “Adjacent”
- “Peak”

Examples:

- Monotonic Array
 - Valid Mountain Array
 - Non-decreasing Array
 - Find Peak Element
-

4.6 Modify Array In-Place

Problem clues:

- “Do it in-place”
- “Do not allocate extra space”
- “Move all...”
- “Remove all...”
- “Return new length...”

Examples:

- Move Zeroes
- Remove Element
- Remove Duplicates from Sorted Array

- Sort Array By Parity
-

4.7 Traverse From Right to Left

Problem clues:

- “Elements to the right”
- “Greatest value on right side”
- “Suffix”
- “Future values”

Examples:

- Replace Elements with Greatest Element on Right Side
 - Product of Array Except Self
 - Suffix maximum variants
-

5. When NOT to Use Only Basic Array Traversal

Basic traversal is not always enough.

5.1 Need Fast Lookup

Example:

Two Sum in an unsorted array.

You need to quickly check whether target - nums[i] exists.

Use hashing.

5.2 Need a Contiguous Window With Dynamic Size

Example:

Longest Substring Without Repeating Characters.

You need to maintain a moving substring.

Use sliding window.

5.3 Need Many Range Queries

Example:

Many queries asking sum from left to right.

Use prefix sum.

5.4 Need Sorted Relationships

Example:

3Sum.

Sorting + two pointers is better.

5.5 Need Top K Values

Example:

Kth Largest Element.

Use heap, quickselect, or sorting.

5.6 Need All Combinations

Example:

Subsets, permutations, N-Queens.

Use backtracking.

5.7 Need Complex Choice Dependencies

Example:

House Robber, Coin Change, Edit Distance.

Use dynamic programming.

6. Core Intuition

At each index, ask four questions:

Question 1

What is the current element?

```
nums[i]
```

Question 2

What useful information do I know so far?

Examples:

```
maxVal  
minVal  
sum  
count  
best  
current  
write  
leftSum  
maxRight
```

Question 3

Can the current element improve the answer?

Example:

```
best = max(best, candidate);
```

Question 4

What should I update for future elements?

Example:

```
minPrice = min(minPrice, prices[i]);
```

A lot of traversal problems follow this structure:

```
initialize state
```

```
for each element:
```

```
    use current element
```

update answer
update state

return answer

7. Mental Models

7.1 Backpack of Information

Imagine walking through the array.
You cannot carry the whole past unless needed.
Instead, you carry only useful information.
For stock profit, your backpack contains:

minPrice
bestProfit

You do not need every previous price.
You only need the cheapest previous price.

7.2 Current Candidate vs Global Best

Many problems separate:

current
best

current means the answer ending here or using the current element.
best means the best answer seen anywhere.

Example:

```
currentSum = max(nums[i], currentSum + nums[i]);  
bestSum = max(bestSum, currentSum);
```

7.3 Read Pointer / Write Pointer

For in-place modification:

- read scans every element
- write marks where to place the next valid element

Example:

```
int write = 0;  
  
for (int read = 0; read < nums.size(); read++) {  
    if (nums[read] != 0) {  
        nums[write] = nums[read];  
        write++;  
    }  
}
```

7.4 Neighbor Comparison

For sorted or monotonic checks:

```
nums[i] compared with nums[i - 1]
```

This is useful for:

- sorted check
- monotonic check
- mountain array
- trend changes

7.5 Right-to-Left Memory

Sometimes current answer depends on future/right-side values.

Then scan from right to left.

Example:

```
int maxRight = -1;

for (int i = n - 1; i >= 0; i--) {
    int current = arr[i];
    arr[i] = maxRight;
    maxRight = max(maxRight, current);
}
```

8. Pattern Recognition Checklist

During an interview, ask:

1. Can I solve this by looking at every element once?
2. Do I only need a few variables?
3. Is the problem asking for count, sum, max, min, or best-so-far?
4. Does each index depend only on previous information?
5. Does each index depend on right-side information?
6. Is this an in-place filtering/removal problem?
7. Do I need index access or only values?
8. Do adjacent values matter?
9. Are there negative numbers or overflow concerns?
10. Is a heavier pattern unnecessary?

If the answer to most of these is yes, start with Basic Array Traversal.

9. C++ Tools Needed

9.1 `vector<int>`

Most LeetCode array inputs use:

```
vector<int> nums
```

Access element:

```
nums[i]
```

Get size:

```
nums.size()
```

9.2 Passing by Reference

Use this when you do not want to copy the array:

```
int solve(vector<int>& nums)
```

For in-place modification, reference is necessary:

```
void moveZeroes(vector<int>& nums)
```

9.3 Index-Based Loop

Use when you need indices:

```
for (int i = 0; i < nums.size(); i++) {  
    // use nums[i]  
}
```

9.4 Range-Based Loop

Use when you only need values:

```
for (int x : nums) {  
    // use x  
}
```

9.5 max() and min()

```
best = max(best, candidate);  
smallest = min(smallest, nums[i]);
```

9.6 long long

Use when sums or products may overflow int.

```
long long sum = 0;
```

9.7 swap()

Useful for in-place operations:

```
swap(nums[i], nums[j]);
```

10. Core Templates

Template 1 — Basic Index Traversal

```
for (int i = 0; i < nums.size(); i++) {  
    int current = nums[i];  
  
    // process current  
}
```

Use when:

- you need index
 - you compare neighbors
 - you modify elements
 - you need position-based logic
-

Template 2 — Basic Value Traversal

```
for (int x : nums) {  
    // process x  
}
```

Use when:

- index does not matter
 - reading values only
 - counting or summing
-

Template 3 — Find Maximum

```
int findMax(vector<int>& nums) {  
    int maxVal = nums[0];  
  
    for (int i = 1; i < nums.size(); i++) {  
        maxVal = max(maxVal, nums[i]);  
    }  
  
    return maxVal;  
}
```

Important:

Do not initialize with 0 unless all values are guaranteed non-negative.

Template 4 — Find Minimum

```
int findMin(vector<int>& nums) {  
    int minVal = nums[0];  
  
    for (int i = 1; i < nums.size(); i++) {  
        minVal = min(minVal, nums[i]);  
    }  
}
```

```
    return minVal;
}
```

Template 5 — Count Elements Matching Condition

```
int countMatching(vector<int>& nums) {
    int count = 0;

    for (int x : nums) {
        if (/* condition */) {
            count++;
        }
    }

    return count;
}
```

Example:

```
if (x % 2 == 0) {
    count++;
}
```

Template 6 — Running Sum

```
long long sumArray(vector<int>& nums) {
    long long sum = 0;

    for (int x : nums) {
        sum += x;
    }

    return sum;
}
```

Template 7 — Running Sum Result Array

```
vector<int> runningSum(vector<int>& nums) {
    vector<int> result(nums.size());

    int sum = 0;

    for (int i = 0; i < nums.size(); i++) {
        sum += nums[i];
        result[i] = sum;
    }
}
```

```
    }  
  
    return result;  
}
```

Template 8 — Track Best Answer So Far

```
int best = initialValue;  
  
for (int i = 0; i < nums.size(); i++) {  
    int candidate = /* compute using nums[i] */;  
  
    best = max(best, candidate);  
}  
  
return best;
```

Template 9 — Compare Neighbors

```
for (int i = 1; i < nums.size(); i++) {  
    if (nums[i] < nums[i - 1]) {  
        // found decreasing pair  
    }  
}
```

Safe because i starts at 1.

Template 10 — Read/Write Pointer

```
int write = 0;  
  
for (int read = 0; read < nums.size(); read++) {  
    if (/* nums[read] should be kept */) {  
        nums[write] = nums[read];  
        write++;  
    }  
}  
  
return write;
```

Used for:

- Remove Element
- Move Zeroes
- Remove Duplicates
- Compacting arrays

Template 11 — Reverse Traversal

```
for (int i = (int)nums.size() - 1; i >= 0; i--) {  
    // process nums[i]  
}
```

Use when current element depends on values to its right.

Template 12 — Build Result Array

```
vector<int> result;  
  
for (int x : nums) {  
    result.push_back(/* transformed value */);  
}  
  
return result;
```

If size is known:

```
vector<int> result(nums.size());  
  
for (int i = 0; i < nums.size(); i++) {  
    result[i] = /* transformed value */;  
}
```

11. Major Variations of Basic Array Traversal

Find max/min	maxVal, minVal
Count values	count
Running sum	sum
Best answer so far	best, candidate
Kadane-style traversal	current, best
Consecutive streak	currentStreak, bestStreak
Neighbor comparison	nums[i], nums[i - 1]
In-place filtering	read, write

Variation	Core State
Right-to-left traversal	maxRight, suffixState
Prefix/suffix style	left info + right info

12. Full Problem Walkthrough 1 — LeetCode 1480: Running Sum of 1D Array

12.1 Problem Summary

Given an array `nums`, return an array `runningSum` where:

```
runningSum[i] = nums[0] + nums[1] + ... + nums[i]
```

Example:

```
nums = {1, 2, 3, 4}
```

Output:

```
{1, 3, 6, 10}
```

12.2 How to Identify This Pattern

The problem says:

- running sum
- cumulative total
- each answer depends on previous sum and current value

This is Basic Traversal / Prefix-style traversal.

12.3 Brute Force Idea

For each index `i`, sum from 0 to `i`.

```
for each i:
    sum nums[0..i]
```

This repeats work.

Time: $O(n^2)$

12.4 Optimized Intuition

Maintain a running sum.

At each index:

```
sum += nums[i];
result[i] = sum;
```

12.5 C++ Code

```

class Solution {
public:
    vector<int> runningSum(vector<int>& nums) {
        vector<int> result(nums.size());

        int sum = 0;

        for (int i = 0; i < nums.size(); i++) {
            sum += nums[i];
            result[i] = sum;
        }

        return result;
    }
};

```

12.6 Code Explanation

```
vector<int> result(nums.size());
```

Creates output array of the same size.

```
int sum = 0;
```

Stores cumulative sum.

```
sum += nums[i];
```

Adds current value.

```
result[i] = sum;
```

Stores running sum at index i.

12.7 Dry Run

Input:

```
nums = {1, 2, 3, 4}
```

0	1	1	{1, _, _, _}
1	2	3	{1, 3, _, _}
2	3	6	{1, 3, 6, _}
3	4	10	{1, 3, 6, 10}

12.8 Complexity

Time: $O(n)$

Each element is processed once.

Space: $O(n)$

The result array stores n values.

12.9 Edge Cases

- Single element
 - Negative values
 - Zeroes
 - Large values, use long long if needed
-

12.10 Similar Problems

- Find the Highest Altitude
 - Find Pivot Index
 - Range Sum Query — Immutable
 - Left and Right Sum Differences
-

13. Full Problem Walkthrough 2 — LeetCode 121: Best Time to Buy and Sell Stock

13.1 Problem Summary

Given prices where $prices[i]$ is the price on day i .

Choose one day to buy and a later day to sell.

Return maximum profit.

Example:

prices = {7, 1, 5, 3, 6, 4}

Output:

5

Buy at 1, sell at 6.

13.2 How to Identify This Pattern

Problem asks:

- best profit
- buy before sell
- one pass possible
- need best previous buying price

This is “track best answer so far.”

13.3 Brute Force Idea

Try all pairs:

```
for buy:
    for sell after buy:
        update best profit
```

Time: $O(n^2)$

13.4 Optimized Intuition

If selling today, the best buy price is the minimum price seen before today.
Maintain:

```
minPrice
bestProfit
```

13.5 C++ Code

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minPrice = prices[0];
        int bestProfit = 0;

        for (int i = 1; i < prices.size(); i++) {
            int currentProfit = prices[i] - minPrice;

            bestProfit = max(bestProfit, currentProfit);

            minPrice = min(minPrice, prices[i]);
        }

        return bestProfit;
    }
};
```

13.6 Code Explanation

```
int minPrice = prices[0];
```

Cheapest price seen so far.

```
int bestProfit = 0;
```

If no profit is possible, answer is 0.

```
int currentProfit = prices[i] - minPrice;
```

Profit if selling today.

```
bestProfit = max(bestProfit, currentProfit);
```

Update answer.

```
minPrice = min(minPrice, prices[i]);
```

Update cheapest price for future sell days.

13.7 Dry Run

```
prices = {7, 1, 5, 3, 6, 4}
```

1	1	7	-6	0	1
2	5	1	4	4	1
3	3	1	2	4	1
4	6	1	5	5	1
5	4	1	3	5	1

Answer: 5

13.8 Correctness Explanation

For each day, if we sell on that day, the best possible buy day must be the cheapest earlier day. Since we maintain the minimum price seen so far, every possible sell day is considered optimally.

13.9 Complexity

Time: $O(n)$

Space: $O(1)$

13.10 Edge Cases

- Prices decreasing: return 0
- One price: return 0
- Best profit at end
- Cheapest price at beginning
- Duplicate prices

13.11 Common Mistakes

Mistake: Sorting prices

Wrong because day order matters.

Mistake: Allowing sell before buy

Scan left to right to preserve time order.

13.12 Similar Problems

- Best Time to Buy and Sell Stock II
- Best Time to Buy and Sell Stock with Cooldown
- Best Sightseeing Pair

- Maximum Subarray
-

14. Full Problem Walkthrough 3 — LeetCode 53: Maximum Subarray

14.1 Problem Summary

Find the contiguous subarray with the largest sum.

Example:

```
nums = {-2, 1, -3, 4, -1, 2, 1, -5, 4}
```

Output:

```
6
```

Best subarray:

```
{4, -1, 2, 1}
```

14.2 How to Identify This Pattern

Problem asks:

- maximum sum
- contiguous subarray
- best answer while scanning

This is Kadane's Algorithm.

14.3 Brute Force Idea

Try every subarray and calculate sum.

Time: $O(n^2)$ or $O(n^3)$, depending on implementation.

14.4 Optimized Intuition

At each index, decide:

11. Extend previous subarray
12. Start new subarray here

```
currentSum = max(nums[i], currentSum + nums[i]);
```

14.5 C++ Code

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int currentSum = nums[0];
        int bestSum = nums[0];

        for (int i = 1; i < nums.size(); i++) {
            currentSum = max(nums[i], currentSum + nums[i]);
        }
    }
};
```

```

        bestSum = max(bestSum, currentSum);
    }

    return bestSum;
}
};

```

14.6 Dry Run

0	-2	-2	-2
1	1	1	1
2	-3	-2	1
3	4	4	4
4	-1	3	4
5	2	5	5
6	1	6	6
7	-5	1	6
8	4	5	6

Answer: 6

14.7 Complexity

Time: $O(n)$

Space: $O(1)$

14.8 Edge Cases

- All negative values
- One element
- All positive values
- Zeroes mixed in

Important:

```
int bestSum = nums[0];
```

not:

```
int bestSum = 0;
```

14.9 Similar Problems

- Maximum Product Subarray
- Maximum Sum Circular Subarray

- Best Time to Buy and Sell Stock
 - Maximum Absolute Sum of Any Subarray
-

15. Full Problem Walkthrough 4 — LeetCode 283: Move Zeroes

15.1 Problem Summary

Move all zeroes to the end while preserving order of non-zero elements.

Example:

```
nums = {0, 1, 0, 3, 12}
```

Output:

```
{1, 3, 12, 0, 0}
```

15.2 How to Identify This Pattern

Problem says:

- move elements
- preserve order
- in-place

This suggests read/write pointer traversal.

15.3 C++ Code

```
class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        int write = 0;

        for (int read = 0; read < nums.size(); read++) {
            if (nums[read] != 0) {
                nums[write] = nums[read];
                write++;
            }
        }

        while (write < nums.size()) {
            nums[write] = 0;
            write++;
        }
    }
};
```

15.4 Dry Run

nums = {0, 1, 0, 3, 12}

First pass:

0	0	skip	0	{0,1,0,3,12}
1	1	write 1 at index 0	1	{1,1,0,3,12}
2	0	skip	1	{1,1,0,3,12}
3	3	write 3 at index 1	2	{1,3,0,3,12}
4	12	write 12 at index 2	3	{1,3,12,3,12}

Fill rest with zero:

{1, 3, 12, 0, 0}

15.5 Complexity

Time: $O(n)$

Space: $O(1)$

15.6 Common Mistakes

- Forgetting to fill remaining positions with zeroes
- Incrementing write for zeroes
- Losing relative order

15.7 Similar Problems

- Remove Element
- Remove Duplicates from Sorted Array
- Sort Array By Parity
- Apply Operations to an Array

16. Full Problem Walkthrough 5 — LeetCode 485: Max Consecutive Ones

16.1 Problem Summary

Given a binary array, return maximum number of consecutive 1s.

Example:

nums = {1, 1, 0, 1, 1, 1}

Output:

3

16.2 How to Identify This Pattern

Problem asks:

- maximum consecutive streak
- scan array once
- reset when condition breaks

Use streak traversal.

16.3 C++ Code

```
class Solution {
public:
    int findMaxConsecutiveOnes(vector<int>& nums) {
        int current = 0;
        int best = 0;

        for (int x : nums) {
            if (x == 1) {
                current++;
            } else {
                current = 0;
            }

            best = max(best, current);
        }

        return best;
    }
};
```

16.4 Dry Run

1	1	1
1	2	2
0	0	2
1	1	2
1	2	2
1	3	3

16.5 Complexity

Time: $O(n)$

Space: $O(1)$

16.6 Edge Cases

- All ones
 - All zeroes
 - Alternating zeroes and ones
 - Single element
-

16.7 Similar Problems

- Max Consecutive Ones II
 - Max Consecutive Ones III
 - Longest Subarray of 1s After Deleting One Element
 - Consecutive Characters
-

17. Full Problem Walkthrough 6 — LeetCode 896: Monotonic Array

17.1 Problem Summary

Return true if array is monotonic increasing or monotonic decreasing.

Example:

```
nums = {1, 2, 2, 3}
```

Output:

```
true
```

17.2 How to Identify This Pattern

Problem asks about overall direction/trend.

Use neighbor comparison.

17.3 C++ Code

```
class Solution {
public:
    bool isMonotonic(vector<int>& nums) {
        bool increasing = true;
        bool decreasing = true;

        for (int i = 1; i < nums.size(); i++) {
            if (nums[i] > nums[i - 1]) {
                decreasing = false;
            }
        }
    }
};
```

```

        if (nums[i] < nums[i - 1]) {
            increasing = false;
        }
    }

    return increasing || decreasing;
}
};

```

17.4 Dry Run

nums = {1, 3, 2}

1	1	3	true	false
2	3	2	false	false

Return false.

17.5 Edge Cases

- All equal values
- One element
- Already increasing
- Already decreasing
- Direction changes once

17.6 Common Mistake

Treating equal values as invalid.

For monotonic arrays, equal adjacent values are allowed.

18. Full Problem Walkthrough 7 — LeetCode 169: Majority Element

18.1 Problem Summary

Return the element appearing more than $n / 2$ times.

Example:

nums = {2, 2, 1, 1, 1, 2, 2}

Output:

2

18.2 How to Identify This Pattern

This can be solved with hashing, sorting, or traversal.

The best traversal solution is Boyer-Moore Voting Algorithm.

Use it when:

- majority element is guaranteed
- need $O(1)$ space

18.3 Core Intuition

Majority element appears more than all other elements combined.

So it can cancel out other elements and still remain.

18.4 C++ Code

```
class Solution {
public:
    int majorityElement(vector<int>& nums) {
        int candidate = 0;
        int count = 0;

        for (int x : nums) {
            if (count == 0) {
                candidate = x;
            }

            if (x == candidate) {
                count++;
            } else {
                count--;
            }
        }

        return candidate;
    }
};
```

18.5 Dry Run

nums = {2, 2, 1, 1, 1, 2, 2}

2	2	1
2	2	2
1	2	1
1	2	0

x	candidate	count
1	1	1
2	1	0
2	2	1

Return 2.

18.6 Complexity

Time: $O(n)$

Space: $O(1)$

18.7 Important Edge Case

If majority is not guaranteed, verify candidate with second pass.

```
int freq = 0;

for (int x : nums) {
    if (x == candidate) {
        freq++;
    }
}

if (freq > nums.size() / 2) {
    return candidate;
}
```

18.8 Similar Problems

- Majority Element II
 - Degree of an Array
 - Check If a Number Is Majority Element in a Sorted Array
-

19. Full Problem Walkthrough 8 — LeetCode 1299: Replace Elements with Greatest Element on Right Side

19.1 Problem Summary

Replace each element with the greatest element among elements to its right.

Last element becomes -1.

Example:

```
arr = {17, 18, 5, 4, 6, 1}
```

Output:

```
{18, 6, 6, 6, 1, -1}
```

19.2 How to Identify This Pattern

Problem says:

- greatest element to the right
- future/right-side information

Scan from right to left.

19.3 C++ Code

```
class Solution {
public:
    vector<int> replaceElements(vector<int>& arr) {
        int maxRight = -1;

        for (int i = (int)arr.size() - 1; i >= 0; i--) {
            int current = arr[i];

            arr[i] = maxRight;

            maxRight = max(maxRight, current);
        }

        return arr;
    }
};
```

19.4 Dry Run

5	1	-1	-1	1
4	6	1	1	6
3	4	6	6	6
2	5	6	6	6
1	18	6	6	18
0	17	18	18	18

19.5 Common Mistake

Wrong:

```
arr[i] = maxRight;  
maxRight = max(maxRight, arr[i]);
```

This uses the overwritten value.

Correct:

```
int current = arr[i];  
arr[i] = maxRight;  
maxRight = max(maxRight, current);
```

20. Full Problem Walkthrough 9 — LeetCode 26: Remove Duplicates from Sorted Array

20.1 Problem Summary

Given a sorted array, remove duplicates in-place and return number of unique elements.

Example:

```
nums = {1, 1, 2}
```

Return:

```
2
```

Modified front:

```
{1, 2}
```

20.2 How to Identify This Pattern

Problem says:

- sorted array
- remove duplicates
- in-place
- return new length

Use read/write pointer.

20.3 C++ Code

```
class Solution {  
public:  
    int removeDuplicates(vector<int>& nums) {  
        if (nums.empty()) {  
            return 0;  
        }  
    }  
};
```

```

    }

    int write = 1;

    for (int read = 1; read < nums.size(); read++) {
        if (nums[read] != nums[write - 1]) {
            nums[write] = nums[read];
            write++;
        }
    }

    return write;
}
};

```

20.4 Why This Works

Since the array is sorted, duplicates are adjacent.
 nums[write - 1] is the last unique value stored.
 If nums[read] differs, it is a new unique value.

20.5 Dry Run

nums = {0, 0, 1, 1, 1, 2, 2, 3}

1	0	0	skip	1
2	1	0	write 1	2
3	1	1	skip	2
4	1	1	skip	2
5	2	1	write 2	3
6	2	2	skip	3
7	3	2	write 3	4

Return 4.

21. Full Problem Walkthrough 10 — LeetCode 27: Remove Element

21.1 Problem Summary

Remove all occurrences of val in-place and return new length.

Example:

```
nums = {3, 2, 2, 3}
val = 3
```

Return:

```
2
```

Front becomes:

```
{2, 2}
```

21.2 How to Identify This Pattern

Problem says:

- remove values
- in-place
- return length

Use read/write pointer.

21.3 C++ Code

```
class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int write = 0;

        for (int read = 0; read < nums.size(); read++) {
            if (nums[read] != val) {
                nums[write] = nums[read];
                write++;
            }
        }

        return write;
    }
};
```

21.4 Complexity

Time: $O(n)$

Space: $O(1)$

21.5 Similar Problems

- Move Zeroes
- Remove Duplicates from Sorted Array
- Remove Duplicates from Sorted Array II
- String Compression

22. Full Problem Walkthrough 11 — LeetCode 152: Maximum Product Subarray

This is a harder extension.

22.1 Problem Summary

Find the contiguous subarray with the largest product.

Example:

```
nums = {2, 3, -2, 4}
```

Output:

```
6
```

Subarray:

```
{2, 3}
```

22.2 How to Identify This Pattern

Problem asks for:

- maximum product
- contiguous subarray
- negative numbers may flip sign

This is similar to Maximum Subarray, but harder because negative values can turn a small negative product into a large positive product.

22.3 Key Insight

Track both:

```
maxProductEndingHere  
minProductEndingHere
```

Why minimum?

Because multiplying by a negative can turn minimum into maximum.

22.4 C++ Code

```
class Solution {  
public:  
    int maxProduct(vector<int>& nums) {  
        int currentMax = nums[0];  
        int currentMin = nums[0];  
        int best = nums[0];  
  
        for (int i = 1; i < nums.size(); i++) {
```

```

    int x = nums[i];

    if (x < 0) {
        swap(currentMax, currentMin);
    }

    currentMax = max(x, currentMax * x);
    currentMin = min(x, currentMin * x);

    best = max(best, currentMax);
}

return best;
}
};

```

22.5 Dry Run

nums = {2, 3, -2, 4}

0	2	2	2	2
1	3	6	3	6
2	-2	-2	-12	6
3	4	4	-48	6

Return 6.

22.6 Edge Cases

- All negative numbers
- Zeroes
- One element
- Negative pair creates positive product

22.7 Similar Problems

- Maximum Subarray
- Maximum Sum Circular Subarray
- Product of Array Except Self

23. Full Problem Walkthrough 12 — LeetCode 605: Can Place Flowers

23.1 Problem Summary

Given a flowerbed with 0s and 1s, determine if n new flowers can be planted without adjacent flowers.

Example:

```
flowerbed = {1, 0, 0, 0, 1}
```

```
n = 1
```

Output:

```
true
```

23.2 How to Identify This Pattern

Problem asks:

- scan array
- check neighbors
- greedily place when possible

Basic traversal + neighbor checks.

23.3 C++ Code

```
class Solution {
public:
    bool canPlaceFlowers(vector<int>& flowerbed, int n) {
        int size = flowerbed.size();

        for (int i = 0; i < size; i++) {
            if (flowerbed[i] == 0) {
                bool leftEmpty = (i == 0 || flowerbed[i - 1] == 0);
                bool rightEmpty = (i == size - 1 || flowerbed[i + 1] == 0);

                if (leftEmpty && rightEmpty) {
                    flowerbed[i] = 1;
                    n--;
                }

                if (n == 0) {
                    return true;
                }
            }
        }

        return n <= 0;
    }
};
```

23.4 Key Idea

At each empty position, check whether both neighbors are empty or out of bounds. If yes, plant greedily.

23.5 Edge Cases

- First index
 - Last index
 - Single-element flowerbed
 - $n = 0$
 - Already full flowerbed
-

24. Mini Problem 1 — LeetCode 1732: Find the Highest Altitude

Pattern

Running sum + max.

```
class Solution {
public:
    int largestAltitude(vector<int>& gain) {
        int altitude = 0;
        int best = 0;

        for (int x : gain) {
            altitude += x;
            best = max(best, altitude);
        }

        return best;
    }
};
```

Key idea:

- Start at altitude 0
 - Each gain changes altitude
 - Track highest altitude seen
-

25. Mini Problem 2 — LeetCode 1672: Richest Customer Wealth

Pattern

Nested traversal.

```

class Solution {
public:
    int maximumWealth(vector<vector<int>>& accounts) {
        int best = 0;

        for (vector<int>& customer : accounts) {
            int sum = 0;

            for (int money : customer) {
                sum += money;
            }

            best = max(best, sum);
        }

        return best;
    }
};

```

Key idea:

- Sum each row
- Track maximum row sum

26. Mini Problem 3 — LeetCode 905: Sort Array By Parity

Pattern

In-place two-pointer/write style.

Simple result-array version:

```

class Solution {
public:
    vector<int> sortArrayByParity(vector<int>& nums) {
        vector<int> result;

        for (int x : nums) {
            if (x % 2 == 0) {
                result.push_back(x);
            }
        }

        for (int x : nums) {
            if (x % 2 != 0) {

```

```

        result.push_back(x);
    }
}

return result;
}
};

```

In-place version:

```

class Solution {
public:
    vector<int> sortArrayByParity(vector<int>& nums) {
        int write = 0;

        for (int read = 0; read < nums.size(); read++) {
            if (nums[read] % 2 == 0) {
                swap(nums[write], nums[read]);
                write++;
            }
        }

        return nums;
    }
};

```

27. Mini Problem 4 — LeetCode 941: Valid Mountain Array

Pattern

Neighbor comparison.

```

class Solution {
public:
    bool validMountainArray(vector<int>& arr) {
        int n = arr.size();

        if (n < 3) {
            return false;
        }

        int i = 0;

```

```
while (i + 1 < n && arr[i] < arr[i + 1]) {  
    i++;  
}  
  
if (i == 0 || i == n - 1) {  
    return false;  
}  
  
while (i + 1 < n && arr[i] > arr[i + 1]) {  
    i++;  
}  
  
return i == n - 1;  
}  
};
```

Key idea:

- Must strictly increase
- Then strictly decrease
- Peak cannot be first or last

28. Common Edge Cases

28.1 Empty Array

Some problems guarantee non-empty input.

If not guaranteed:

```
if (nums.empty()) {  
    return 0;  
}
```

Important before:

```
nums[0]
```

28.2 Single Element

Examples:

```
nums = {5}
```

Check:

- maximum should be 5
- maximum subarray should be 5
- stock profit should be 0
- monotonic array should be true

28.3 All Negative Values

Important for max problems.

Wrong:

```
int best = 0;
```

Correct:

```
int best = nums[0];
```

28.4 All Zeroes

Important for:

- Move Zeroes
- Max Consecutive Ones
- Product problems
- Count problems

28.5 Duplicates

Important for:

- Remove duplicates
- Majority element
- Monotonic array
- Counting

28.6 Already Sorted Array

Neighbor comparison should handle this.

28.7 Reverse Sorted Array

Stock profit returns 0.

Monotonic decreasing returns true.

28.8 Integer Overflow

For sums/products, use:

```
long long sum = 0;
```

or:

```
long long product = 1;
```

28.9 Out-of-Bounds Neighbor Access

Wrong:

```
for (int i = 0; i < nums.size(); i++) {  
    if (nums[i] > nums[i + 1]) {  
        // invalid at last index  
    }  
}
```

Correct:

```
for (int i = 0; i + 1 < nums.size(); i++) {  
    if (nums[i] > nums[i + 1]) {  
        // safe  
    }  
}
```

or:

```
for (int i = 1; i < nums.size(); i++) {  
    if (nums[i] > nums[i - 1]) {  
        // safe  
    }  
}
```

29. Common Mistakes and Wrong Code

Mistake 1 — Wrong Initialization

Wrong:

```
int maxVal = 0;
```

Fails for:

```
nums = {-5, -2, -9}
```

Correct:

```
int maxVal = nums[0];
```

Mistake 2 — Forgetting Empty Check

Wrong:

```
int maxVal = nums[0];
```

If nums can be empty, this crashes.

Mistake 3 — Confusing current and best

For Maximum Subarray:

Wrong:

```
best += nums[i];
```

Correct:

```
current = max(nums[i], current + nums[i]);  
best = max(best, current);
```

Mistake 4 — Updating State in Wrong Order

For Replace Elements:

Wrong:

```
arr[i] = maxRight;  
maxRight = max(maxRight, arr[i]);
```

Correct:

```
int current = arr[i];  
arr[i] = maxRight;  
maxRight = max(maxRight, current);
```

Mistake 5 — Sorting When Order Matters

Wrong for stock profit:

```
sort(prices.begin(), prices.end());
```

Day order matters.

Mistake 6 — Not Preserving Relative Order

For Move Zeroes, swapping with the end may break order.

Read/write pointer preserves order.

30. Interview Explanation Templates

30.1 For Max/Min

I scan the array once and maintain the best value seen so far. For each element, I compare it with the current best and update if needed. This gives $O(n)$ time and $O(1)$ space.

30.2 For Counting

I maintain a counter and increment it whenever the current element satisfies the condition. Since each element is checked once, the time complexity is $O(n)$.

30.3 For Running Sum

I maintain a cumulative sum as I scan. At each index, I add the current value and either store it or use it to update the answer.

30.4 For Best Answer So Far

I scan from left to right and maintain the useful previous information needed to evaluate the current candidate. Then I update the global best answer.

30.5 For Write Pointer

I use a read pointer to scan all elements and a write pointer to place the next valid element. This lets me modify the array in-place while preserving order.

30.6 For Right-to-Left Traversal

Since each element depends on values to its right, I scan from right to left and maintain the best information seen so far on the right side.

31. Complexity Patterns

Find max/min	$O(n)$	$O(1)$
Count values	$O(n)$	$O(1)$
Running sum	$O(n)$	$O(1)$, or $O(n)$ output
Best-so-far	$O(n)$	$O(1)$
Kadane's algorithm	$O(n)$	$O(1)$
Move zeroes	$O(n)$	$O(1)$
Remove element	$O(n)$	$O(1)$
Neighbor comparison	$O(n)$	$O(1)$
Right-to-left scan	$O(n)$	$O(1)$

32. Time-Boxed Practice Guidance

For this chapter, use this timing standard.

Easy traversal problems

Target time:

5–10 minutes

Examples:

- Running Sum
 - Highest Altitude
 - Richest Customer Wealth
-

Core interview easy problems

Target time:

10–15 minutes

Examples:

- Move Zeroes
 - Best Time to Buy and Sell Stock
 - Max Consecutive Ones
 - Monotonic Array
-

Medium traversal extensions

Target time:

20–30 minutes

Examples:

- Maximum Subarray
 - Maximum Product Subarray
 - Non-decreasing Array
 - Increasing Triplet Subsequence
-

33. Expanded Practice Set

33.1 Must-Do Easy

- 1.1. Running Sum of 1D Array
 - 2.2. Build Array from Permutation
 - 3.3. Concatenation of Array
 - 4.4. Richest Customer Wealth
 - 5.5. Find the Highest Altitude
 - 6.6. Find Numbers with Even Number of Digits
 - 7.7. Count Items Matching a Rule
 - 8.8. Shuffle the Array
 - 9.9. Max Consecutive Ones
 - 10.10. Move Zeroes
 - 11.11. Remove Element
 - 12.12. Remove Duplicates from Sorted Array
 - 13.13. Monotonic Array
 - 14.14. Replace Elements with Greatest Element on Right Side
 - 15.15. Sort Array By Parity
 - 16.16. Can Place Flowers
 - 17.17. Valid Mountain Array
-

33.2 Must-Do Core Interview

- 18.18. Best Time to Buy and Sell Stock
- 19.19. Maximum Subarray
- 20.20. Majority Element
- 21.21. Find Pivot Index
- 22.22. Product of Array Except Self
- 23.23. Rotate Array
- 24.24. Non-decreasing Array
- 25.25. Increasing Triplet Subsequence

Some of these combine with later patterns, but they are excellent traversal practice.

33.3 Medium / Harder Extensions

- 26.26. Maximum Product Subarray
- 27.27. Maximum Sum Circular Subarray
- 28.28. Longest Turbulent Subarray
- 29.29. Longest Mountain in Array
- 30.30. Best Sightseeing Pair
- 31.31. Maximum Length of Subarray With Positive Product
- 32.32. Trapping Rain Water, later two pointers / stack

33.33.	Candy, later greedy
34.34.	First Missing Positive, advanced in-place array manipulation

34. Suggested Study Order

Step 1 — Simple Traversal

Start with:

13. Running Sum of 1D Array
14. Richest Customer Wealth
15. Find the Highest Altitude

Goal:

- Become comfortable scanning arrays.
 - Practice sum/count/max updates.
-

Step 2 — Counting and Streaks

Then solve:

16. Find Numbers with Even Number of Digits
17. Max Consecutive Ones
18. Count Items Matching a Rule

Goal:

- Learn counters and reset logic.
-

Step 3 — In-Place Modification

Solve:

19. Move Zeroes
20. Remove Element
21. Remove Duplicates from Sorted Array
22. Sort Array By Parity

Goal:

- Master read/write pointer.
-

Step 4 — Best Answer So Far

Solve:

23. Best Time to Buy and Sell Stock
24. Maximum Subarray
25. Majority Element
26. Best Sightseeing Pair

Goal:

- Learn current candidate vs global best.
-

Step 5 — Neighbor Comparison

Solve:

27. Monotonic Array
28. Valid Mountain Array
29. Non-decreasing Array
30. Increasing Triplet Subsequence

Goal:

- Learn adjacent comparison and trend tracking.

Step 6 — Harder Traversal Extensions

Solve:

31. Maximum Product Subarray
32. Maximum Sum Circular Subarray
33. Longest Turbulent Subarray
34. Longest Mountain in Array

Goal:

- Prepare for DP/greedy-style traversal.
-

35. Pattern Decision Table

Find largest	maxVal	Left to right	Max template
Find smallest	minVal	Left to right	Min template
Count values	count	Left to right	Count template
Running sum	sum	Left to right	Sum template
Max profit	minPrice, bestProfit	Left to right	Best-so-far
Max subarray	currentSum, bestSum	Left to right	Kadane
Consecutive ones	current, best	Left to right	Streak reset
Check monotonic	increasing, decreasing	Left to right	Neighbor compare
Remove values	read, write	Left to right	Write pointer
Right-side max	maxRight	Right to left	Reverse traversal
Product subarray	currentMax, currentMin	Left to right	Dual-state traversal

36. Final Cheat Sheet

Basic Traversal

```
for (int i = 0; i < nums.size(); i++) {  
    // process nums[i]  
}
```

Find Maximum

```
int maxVal = nums[0];  
  
for (int i = 1; i < nums.size(); i++) {  
    maxVal = max(maxVal, nums[i]);  
}
```

Count Condition

```
int count = 0;  
  
for (int x : nums) {  
    if (/* condition */) {  
        count++;  
    }  
}
```

Running Sum

```
long long sum = 0;  
  
for (int x : nums) {  
    sum += x;  
}
```

Best Answer So Far

```
int best = initialValue;  
  
for (int i = 0; i < nums.size(); i++) {  
    int candidate = /* compute candidate */;  
    best = max(best, candidate);  
}
```

Stock Profit

```
int minPrice = prices[0];
int bestProfit = 0;

for (int i = 1; i < prices.size(); i++) {
    bestProfit = max(bestProfit, prices[i] - minPrice);
    minPrice = min(minPrice, prices[i]);
}
```

Kadane's Algorithm

```
int currentSum = nums[0];
int bestSum = nums[0];

for (int i = 1; i < nums.size(); i++) {
    currentSum = max(nums[i], currentSum + nums[i]);
    bestSum = max(bestSum, currentSum);
}
```

Write Pointer

```
int write = 0;

for (int read = 0; read < nums.size(); read++) {
    if (/* keep nums[read] */) {
        nums[write] = nums[read];
        write++;
    }
}
```

Reverse Traversal

```
for (int i = (int)nums.size() - 1; i >= 0; i--) {
    // process from right
}
```

37. What to Master Before Chapter 2

Before moving to Hashing / Hash Map, make sure you can:

- Write loops over vectors
- Track maximum and minimum
- Count elements satisfying a condition

- Maintain running sum
 - Use long long when needed
 - Track best answer so far
 - Explain Kadane's Algorithm
 - Use read/write pointer
 - Modify arrays in-place
 - Compare neighboring elements safely
 - Traverse from right to left
 - Handle empty and single-element arrays
 - Avoid out-of-bounds access
 - Explain $O(n)$ time and $O(1)$ space
-

38. Core Takeaway

Basic Array Traversal is not "just looping."

It is the skill of deciding:

What should I remember while scanning?

Most problems in this chapter follow this pattern:

initialize state

for each element:

 update current state

 update answer

return answer

If you master this chapter, later patterns become easier because they all build on traversal.

Hashing, sliding window, prefix sum, greedy, DP, graph BFS, and monotonic stack all depend on the same foundation:

Process elements carefully, maintain the right state, and update the answer at the right time.
